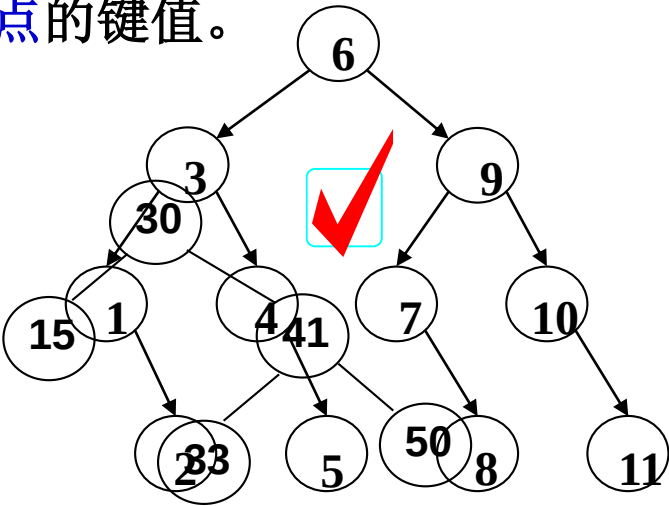
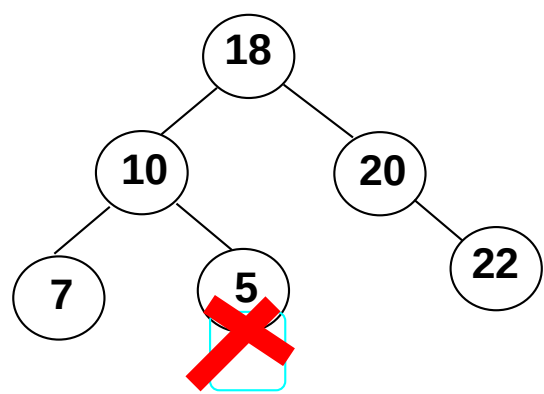


❖ 二叉搜索树

“**二叉搜索树**（**BST, Binary Search Tree**）”也称**二叉排序树**或**二叉查找树**，它是一种对排序和查找都很有用的特殊二叉树。

【定义】 一个二叉搜索树是一棵二叉树，它可以为空。如果不为空，它将满足以下性质：

- 1. 非空**左子树**的所有**键值**小于其**根结点**的键值。
- 2. 非空**右子树**的所有**键值**大于其**根结点**的键值。
- 3. 左、右子树都是**二叉搜索树**。



11个元素**二分查找**的判定树

❖ 二叉搜索树的动态查找

➤ 二叉搜索树作为抽象数据结构的定义与普通二叉树相同，但操作集中多了下列几个特别的函数：



Position Find(ElementType X, BinTree BST)：从二叉搜索树BST中查找元素X，返回其所在结点的地址



Position FindMin(BinTree BST)：从二叉搜索树BST中查找并返回最小元素所在结点的地址



Position FindMax(BinTree BST)：从二叉搜索树BST中查找并返回最大元素所在结点的地址



BinTree Insert(ElementType X, BinTree BST)



BinTree Delete(ElementType X, BinTree BST)

❖ 二叉搜索树的查找操作Find

➤ 查找从根结点开始，如果树为空，没有找到关键字为X的结点。ElementType 是否应当先考察空树？

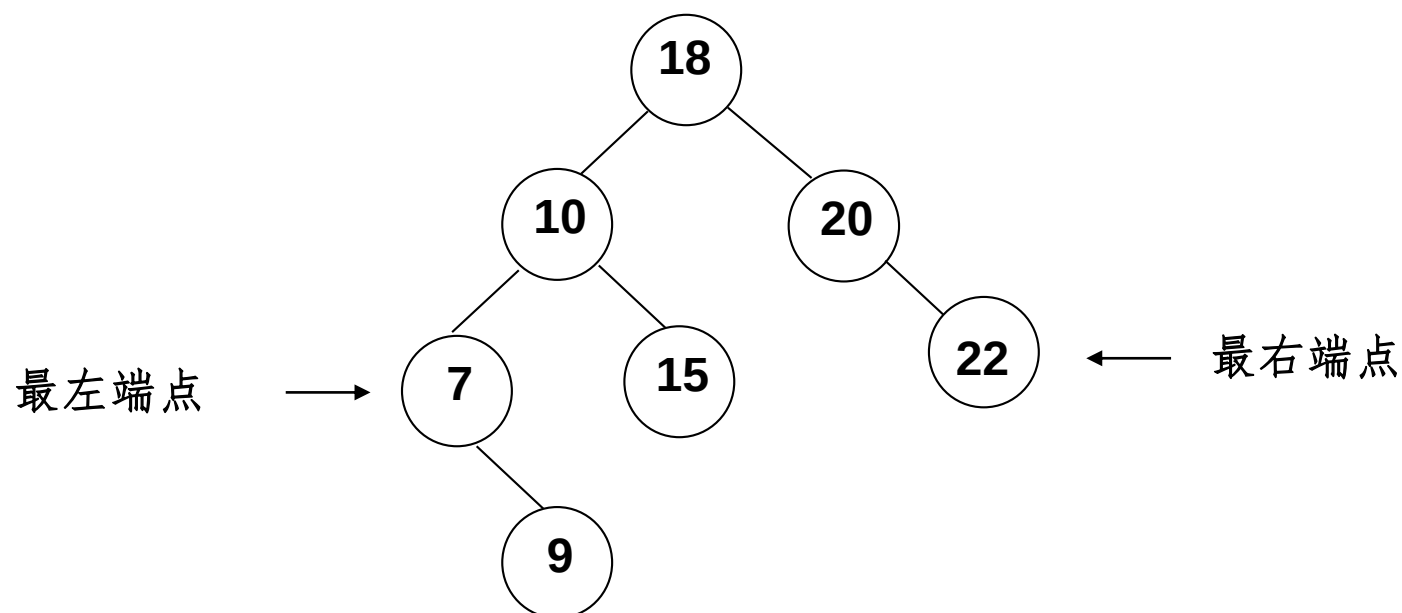
```
{
➤ 若搜索树非空，则根结点关键字和X比较，
  if ( !BST ) return NULL; /* 都是“尾递归”
  需要进行不同的处理：
    if ( X > BST->Data )
    ① 若根结点键值小于X，满足条件的结点将不会出现在它的左子树，
      return Find( X, BST->Right ); /* 在右子树中继续查找 */
      接下来的搜索只需在右子树中进行；
    Else if ( X < BST->Data )
    ② 如果根结点的键值大于X，接下来的搜索将在左子树中进行；
    return Find( X, BST->Left ); /* 在左子树中继续查找 */
    ③ 若两者比较结果是相等，搜索完成，返回指向此结点的指针。
      else /* X == BST->Data */
        return BST; /* 查找成功，返回找到的结点的地址 */
}
```

- 由于非递归函数的执行效率高，一般采用非递归的迭代来实现查找。
- 很容易将“尾递归”函数改为迭代函数

```
Position IterFind( ElementType X, BinTree BST )
{
    while( BST ) {
        if( X > BST->Data )
            BST = BST->Right; /*向右子树中移动，继续查找*/
        else if( X < BST->Data )
            BST = BST->Left; /*向左子树中移动，继续查找*/
        else /* X == BST->Data */
            return BST; /*查找成功，返回找到的结点的地址*/
    }
    return NULL; /*查找失败*/
}
```

❖ 查找最大和最小元素

- 最大元素一定是在树的最右分枝的端结点上
- 最小元素一定是在树的最左分枝的端结点上



```
Position FindMin( BinTree BST )
{
    if( !BST ) return NULL; /*空的二叉搜索树, 返回NULL*/
    else if( !BST->Left )
        return BST; /*找到最左叶结点并返回*/
    else
        return FindMin( BST->Left ); /*沿左分支继续查找*/
}
```

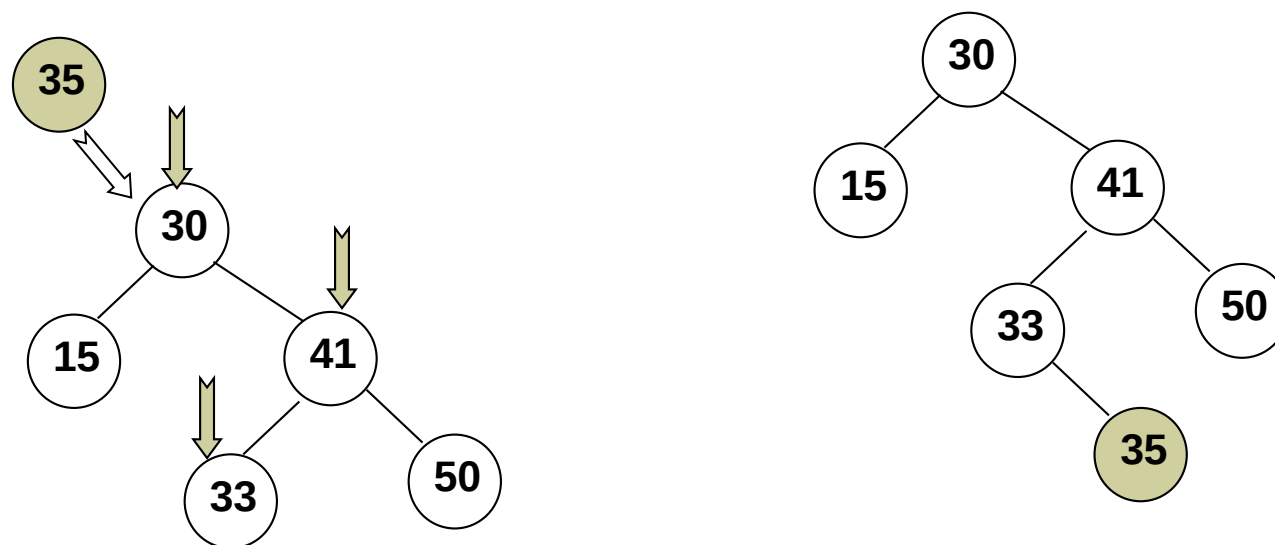
代码4.1 查找最小元素的递归函数

```
Position FindMax( BinTree BST )
{
    if( !BST )
        while( BST->Right ) BST = BST->Right;
        /*沿右分支继续查找, 直到最右叶结点*/
    return BST;
}
```

代码4.2 查找最大元素的迭代函数

❖ 二叉搜索树的插入

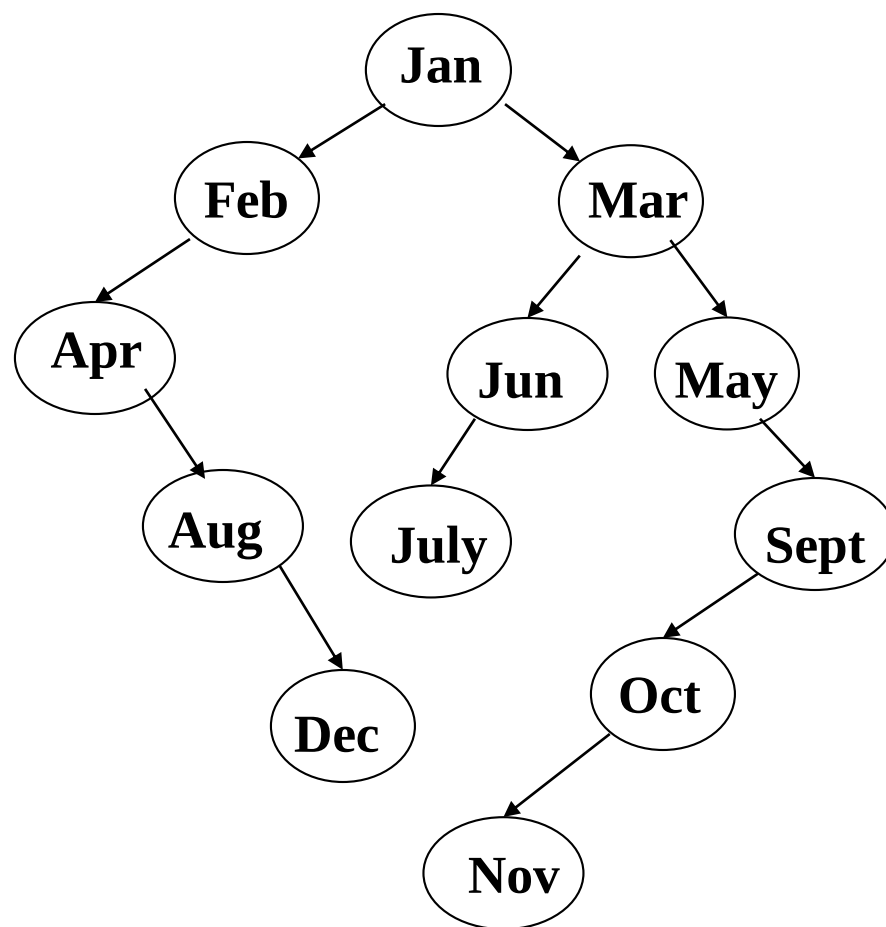
【分析】将元素X插入二叉搜索树BST中，关键是要找到元素应该插入的**位置**。位置的确定可以利用与查找函数**Find**类似的方法，如果在树**BST中找到X**，说明要插入的元素已存在，可放弃插入操作。如果**没找到X**，查找终止的位置就是X应插入的位置。



```
BinTree Insert( ElementType X, BinTree BST )
{
    if( !BST ){
        /*若原树为空，生成并返回一个结点的二叉搜索树*/
        BST = malloc(sizeof(struct TreeNode));
        BST->Data = X;
        BST->Left = BST->Right = NULL;
    }else /*开始找要插入元素的位置*/
        if( X < BST->Data )
            BST->Left = Insert( X, BST->Left);
            /*递归插入左子树*/
        else if( X > BST->Data )
            BST->Right = Insert( X, BST->Right);
            /*递归插入右子树*/
        /* else x已经存在，什么都不做 */
    return BST;
}
```

代码4.3 二叉搜索树的插入算法

【例4.8】以一年十二个月的英文缩写为键值，按从一月到十二月顺序输入它们，即输入序列为（**Jan, Feb, Mar, Apr, May, Jun, July, Aug, Sep, Oct, Nov, Dec**），将产生什么样的二叉搜索树？



❖ 二叉搜索树的删除

➤ 二叉搜索树的删除操作比其它操作更为复杂，有**三种情况**需要考虑：

👉 要删除的是**叶结点**——可以直接删除，然后再修改其父结点的指针。

【例】：删除 35

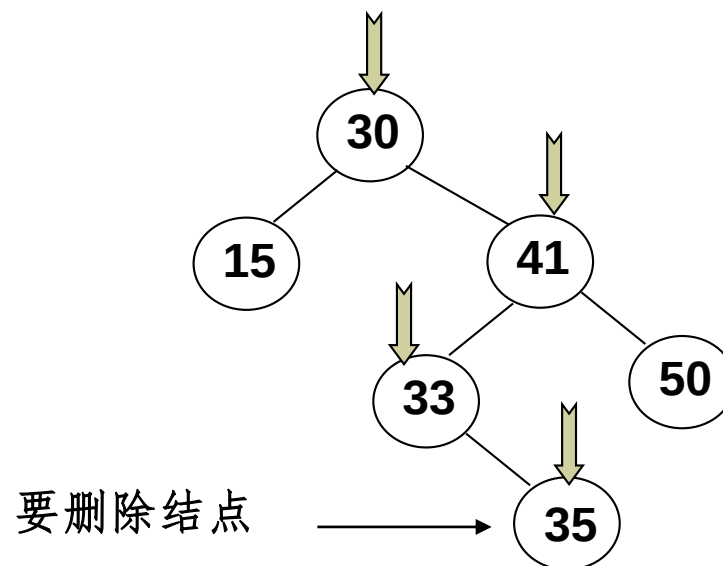


图4.1 二叉搜索树叶结点的删除

☞ 要删除的结点**只有一个孩子**结点——删除之前需要改变其**父结点**的指针，**指向**要删除结点的**孩子结点**。

【例】：删除 33

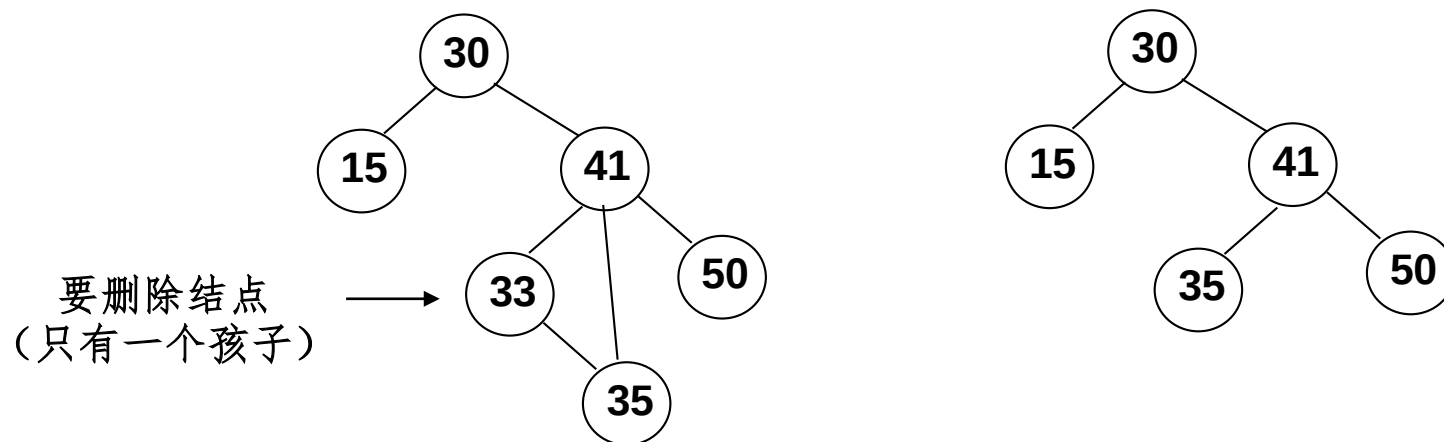
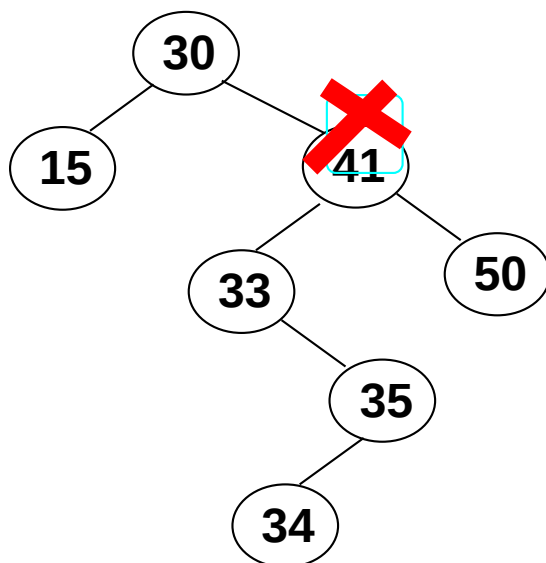


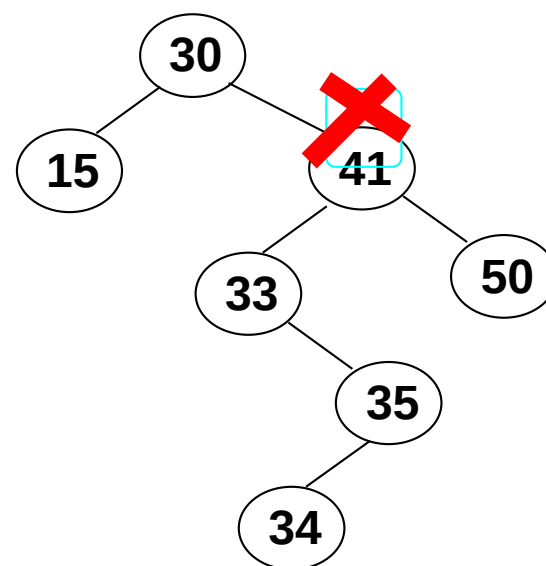
图4.2 具有一个子树的结点删除

☞ 要删除的结点有左、右两棵子树——为了保持二叉搜索树的有序性，替代被删除的元素的位置可以有两种选择：一种是取其右子树中的最小元素；另一个是取其左子树中的最大元素。

【例】：删除 41



(a) 取右子树中的最小元素替代



(b) 取左子树中的最大元素替代

图4.3 具有两个子树的结点删除

```

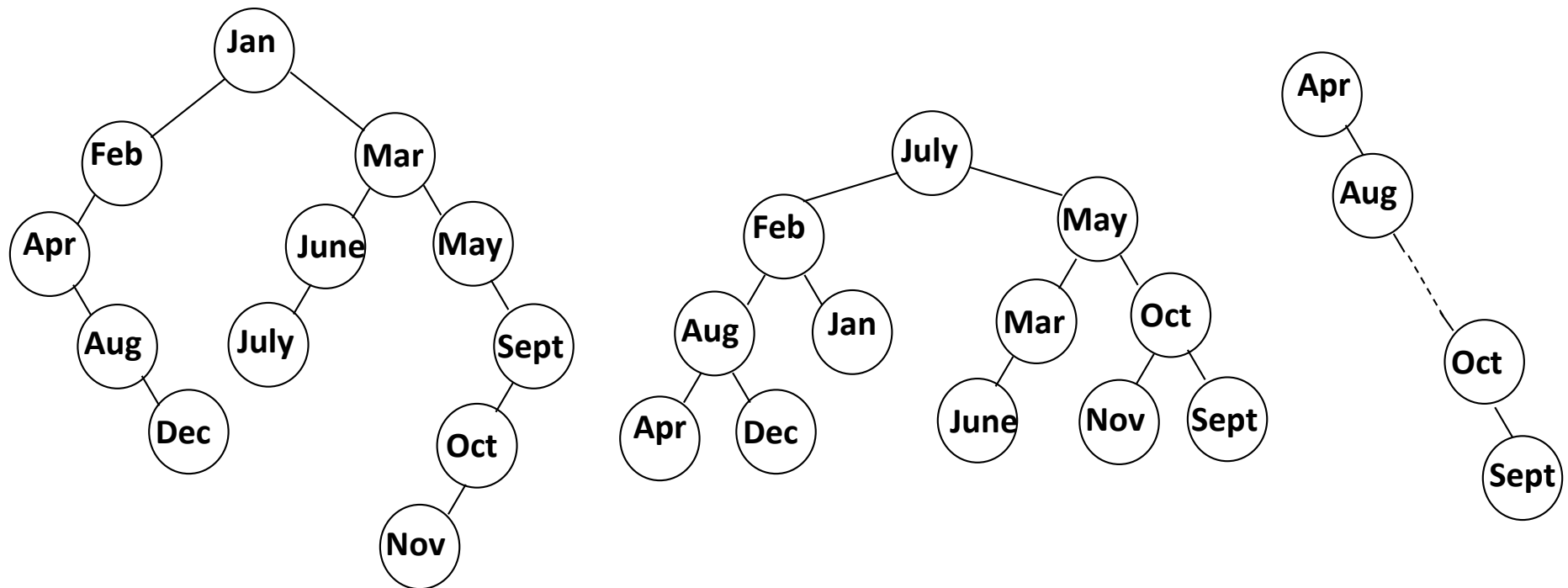
BinTree Delete( ElementType X, BinTree BST )
{
    Position Tmp;
    if( !BST ) printf("要删除的元素未找到");
    else if( X < BST->Data )
        BST->Left = Delete( X, BST->Left); /* 左子树递归删除 */
    else if( X > BST->Data )
        BST->Right = Delete( X, BST->Right); /* 右子树递归删除 */
    else /*找到要删除的结点 */
        if( BST->Left && BST->Right ) { /*被删除结点有左右两个子结点 */
            Tmp = FindMin( BST->Right );
            /*在右子树中找最小的元素填充删除结点*/
            BST->Data = Tmp->Data;
            BST->Right = Delete( BST->Data, BST->Right);
            /*在删除结点的右子树中删除最小元素*/
        } else { /*被删除结点有一个或无子结点*/
            Tmp = BST;
            if( !BST->Left ) /* 有右孩子或无子结点*/
                BST = BST->Right;
            else if( !BST->Right ) /*有左孩子或无子结点*/
                BST = BST->Left;
            free( Tmp );
        }
    return BST;
}

```

❖平衡二叉树

查找二叉搜索树（BST）的时间复杂度（最坏情况下）用查找过程中的比较次数来衡量，它取决于树的深度。

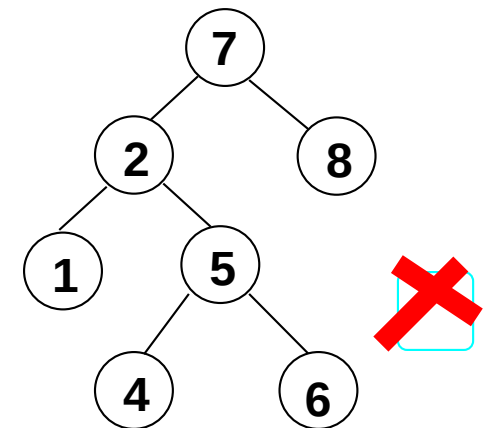
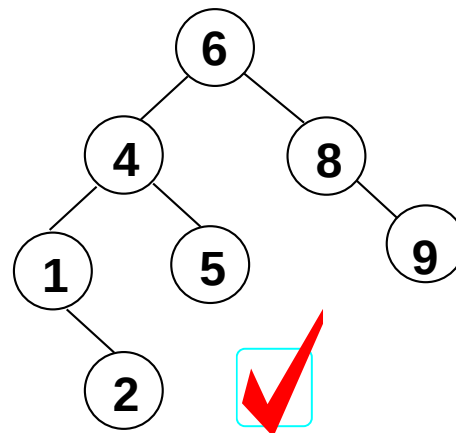
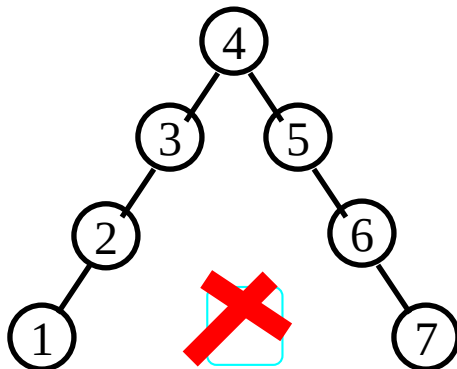
- ❖例
- (a) 按 ASCII 值从小到大排列的月份字符串
- (b) 按 ASCII 值从大到小排列的月份字符串
- (c) 输入序列则是按月份字符串从小到大的顺序排列的。
$$(5 \times 1 + 10 \times 1 + 11 \times 1 + 12 \times 1) / 12 = 6.5;$$



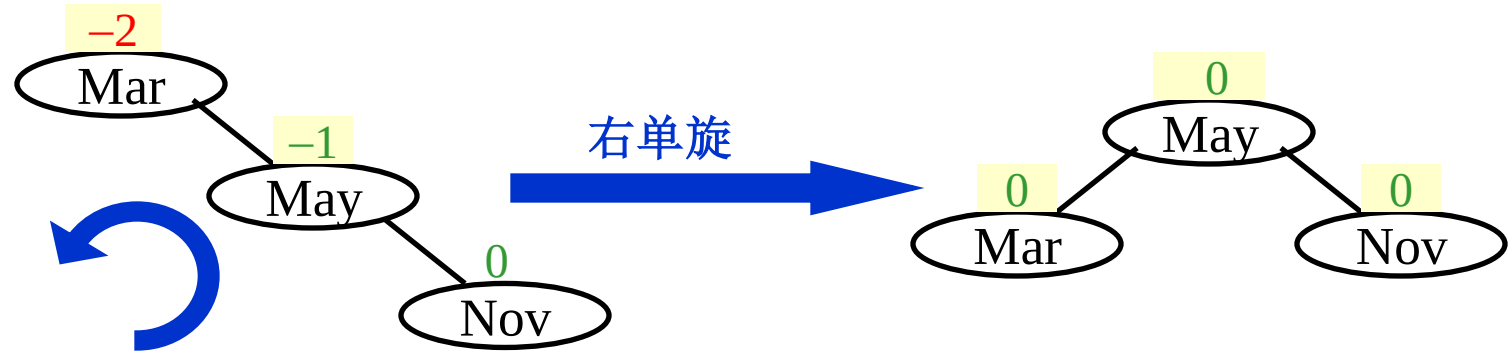
❖ 平衡二叉树的定义

【定义】对于二叉树中任一结点T，其“**平衡因子**（Balance Factor，简称BF）”定义为 $BF(T) = h_L - h_R$ ，其中 h_L 和 h_R 分别为T的左、右子树的高度。

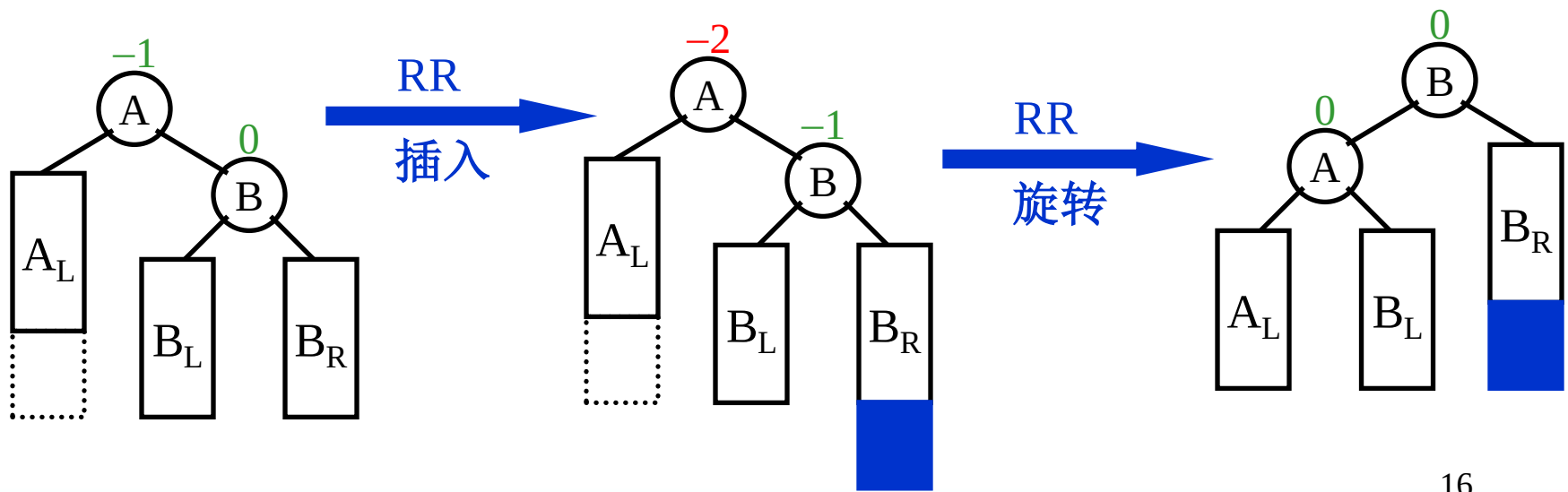
“**平衡二叉树**（Balanced Binary Tree）”又称为“**AVL树**”。AVL树或者是一棵空树，或者是具有下列性质的非空**二叉搜索树**：“任一结点左右子树高度差的绝对值不超过1。”即 $|BF(T)| \leq 1$ 。

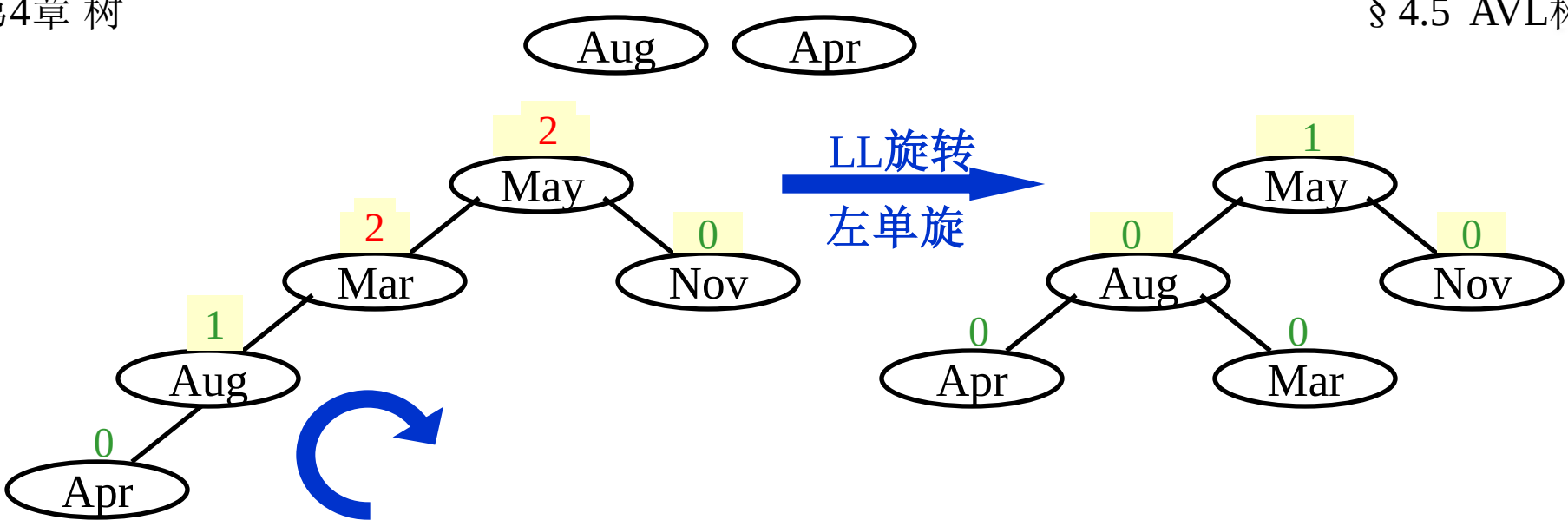


❖ 平衡二叉树的调整

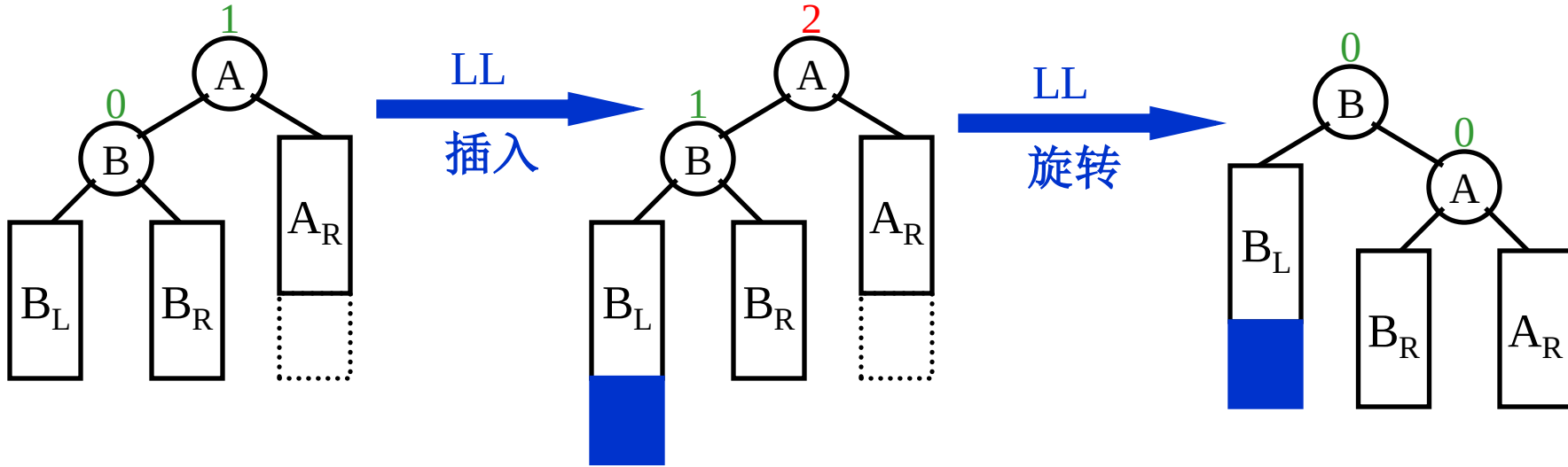


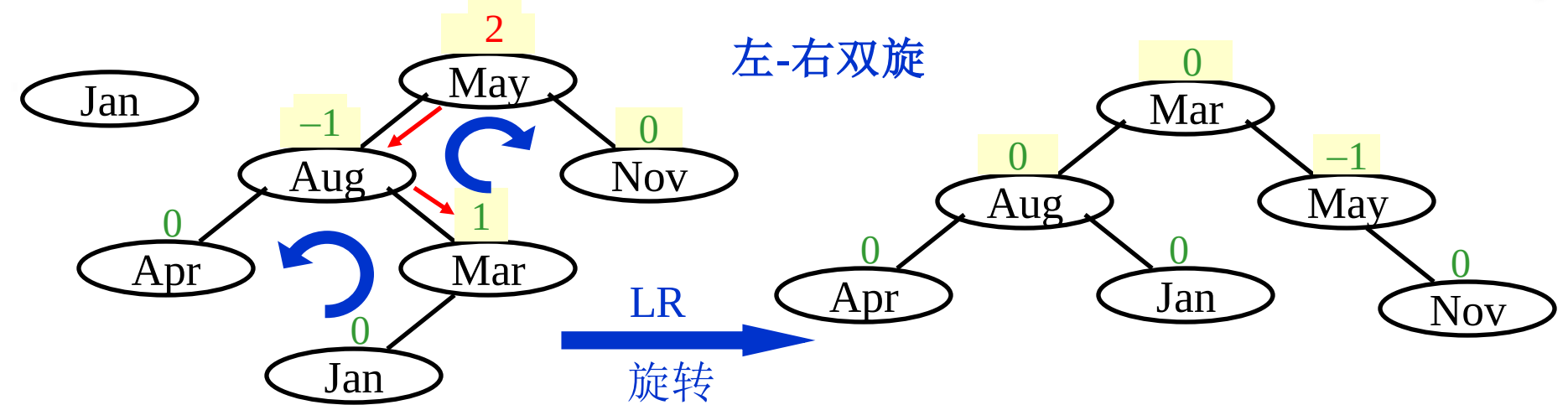
👁️ 首个不平衡的“发现者”是Mar（未必是根结点），它是调整起点位置。
而“麻烦结点”Nov在发现者右子树的右边，因而叫RR插入，需要RR旋转（右单旋）；一般情况（设A是首个发现者）的调整方式如下：



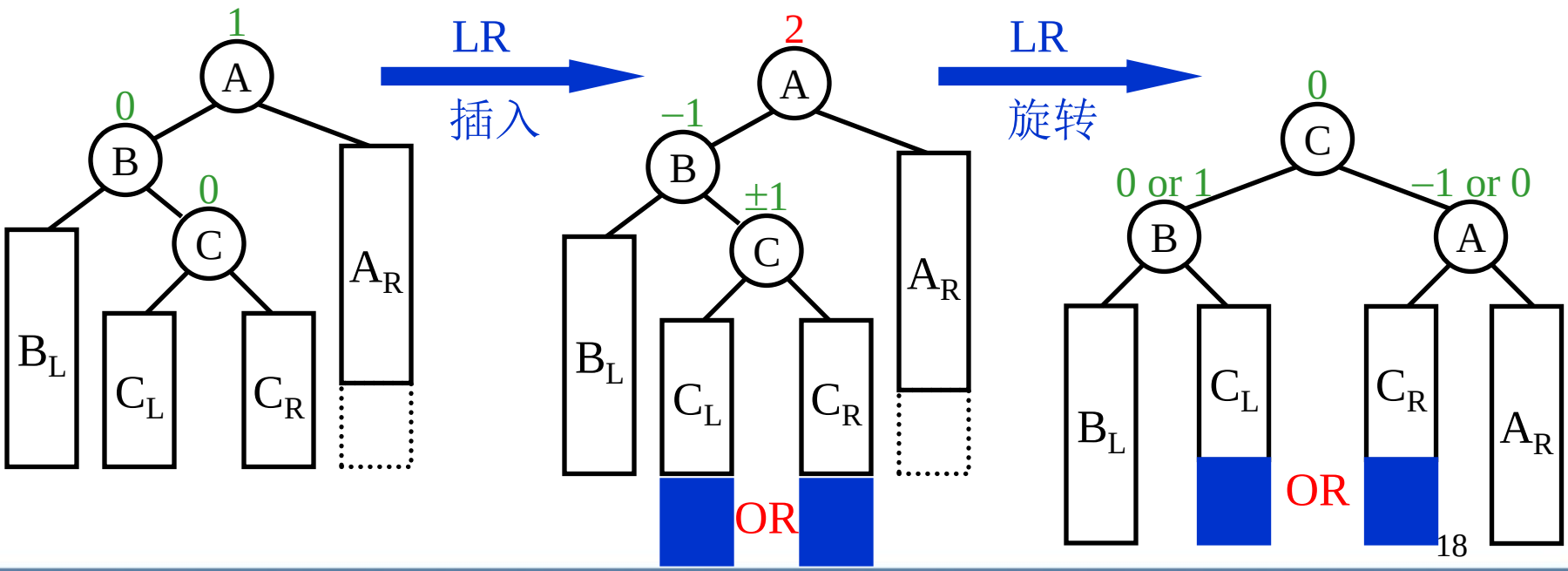


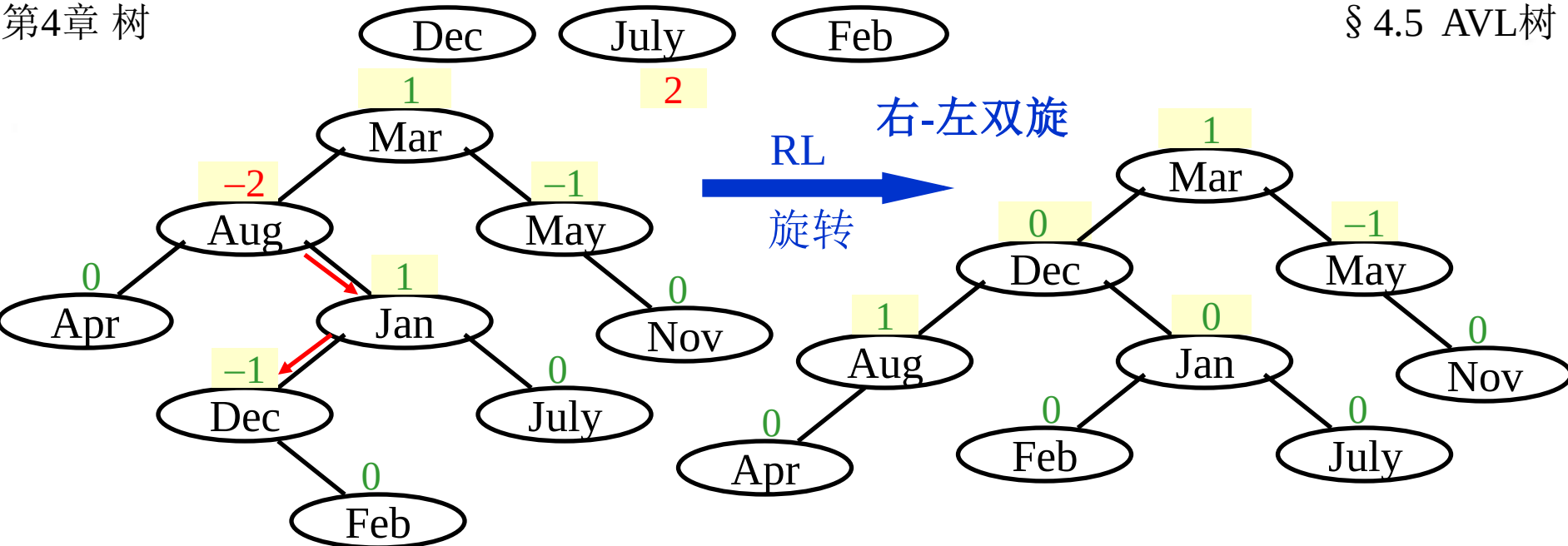
👉 首个“发现者”是Mar；“麻烦结点”Apr在发现者左子树的左边，因而叫LL插入；一般情况（设A是首个发现者）的调整方式如下：



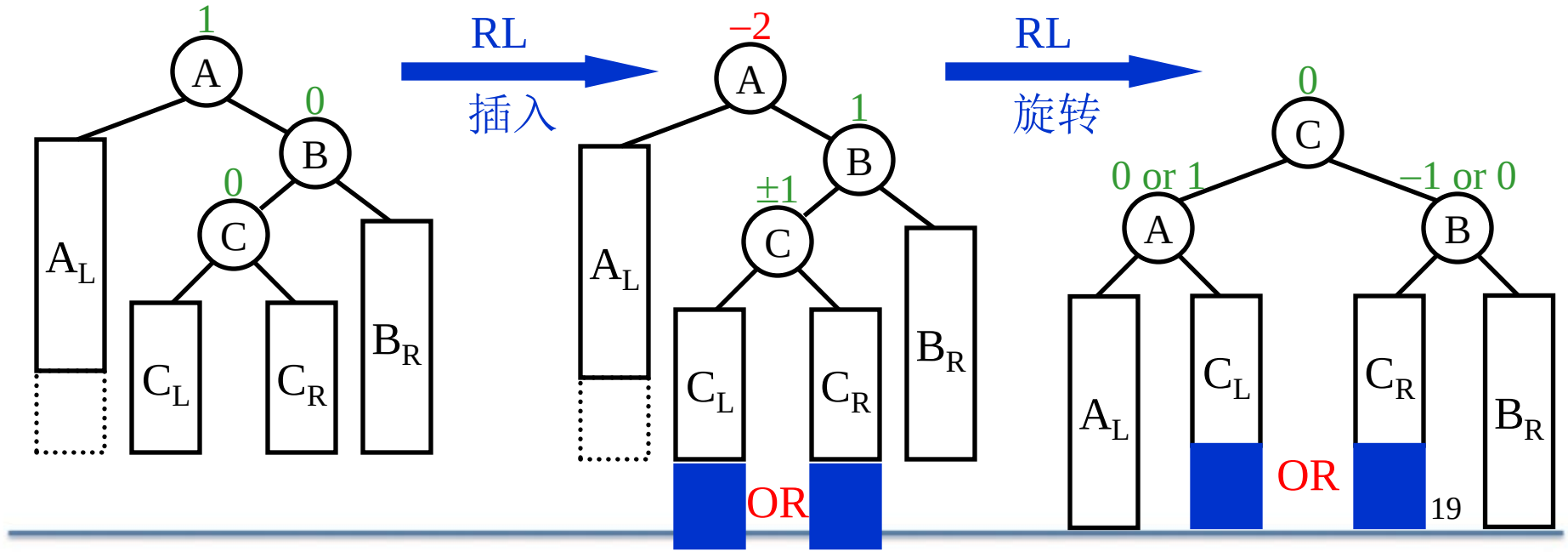


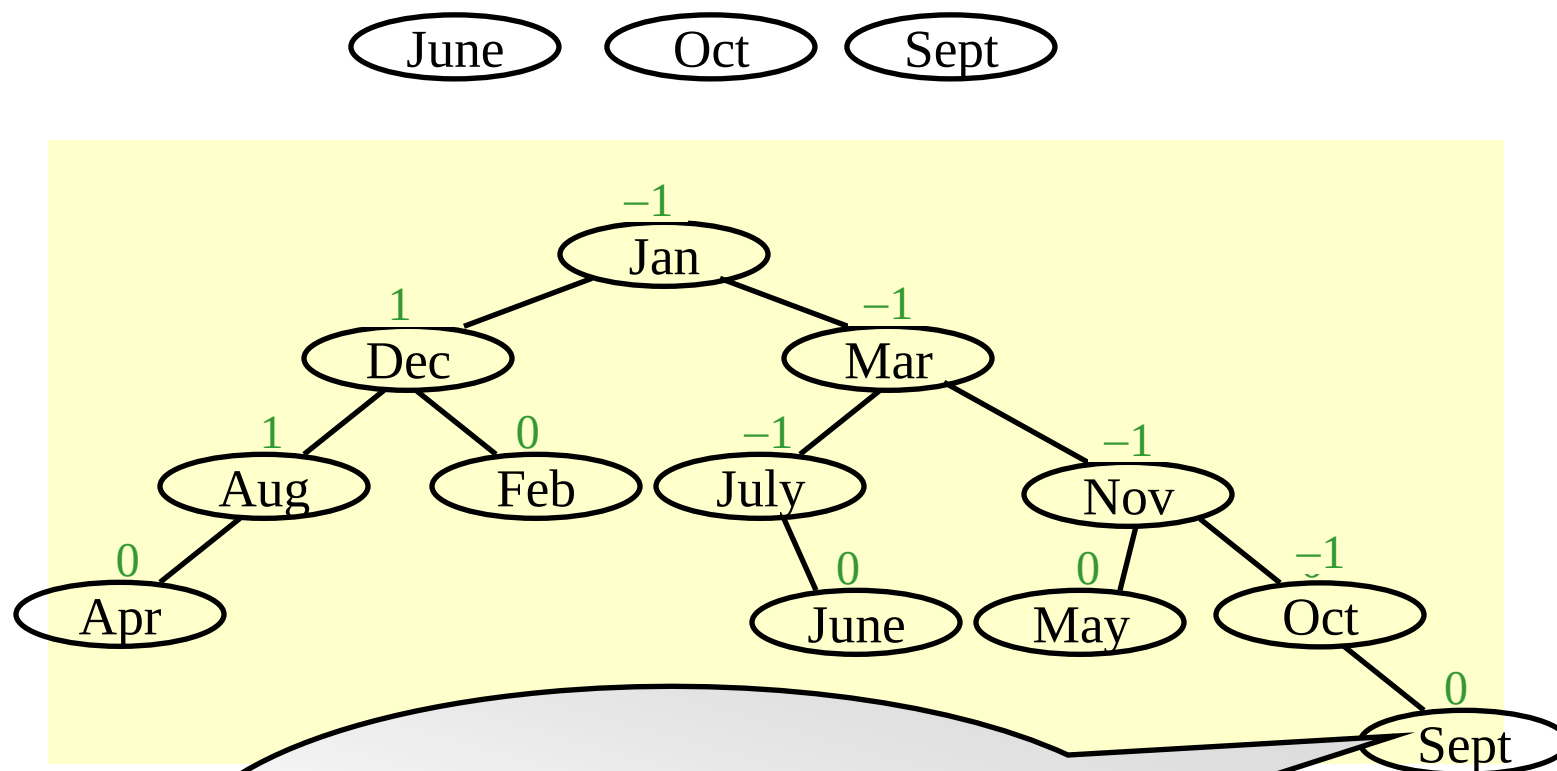
👉 首个“发现者”是May；“麻烦结点”Jan在左子树的右边，因而叫LR 插入；一般情况（设A是首个发现者）的调整如下：





👉 一般情况调整如下:

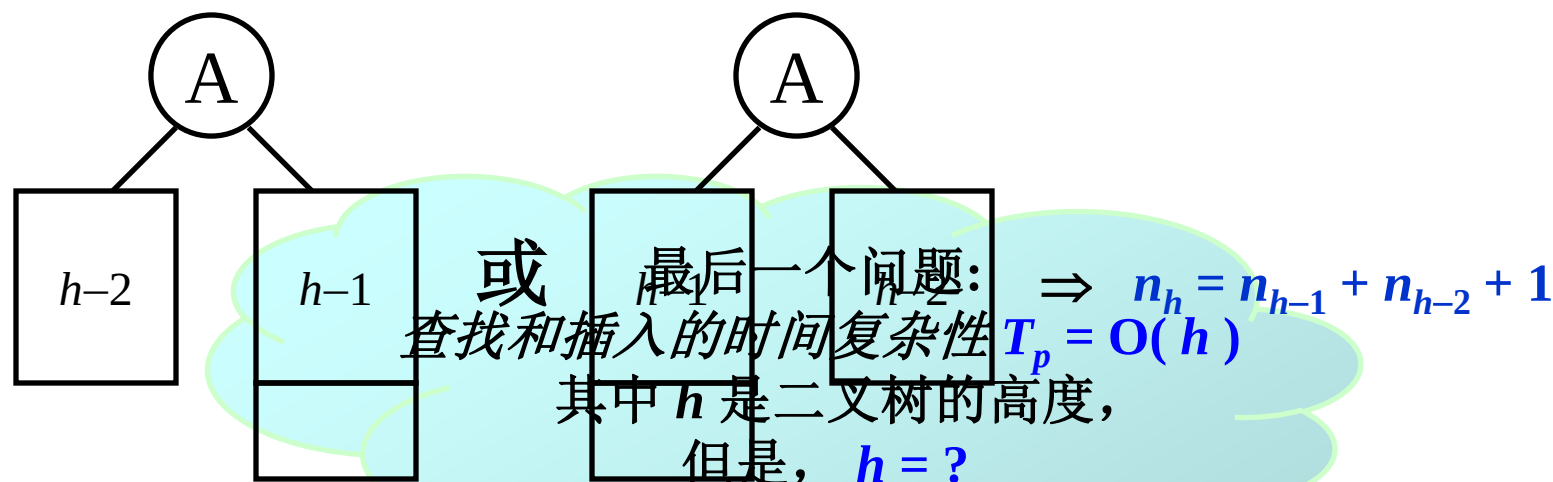




注意：有时候插入元素即便不需要调整结构，也可能需要重新计算一些平衡因子。

LL、RR、LR、RL四种不平衡情况及其它们的旋转调整算法程序，见教材p.131代码4.20—代码4.22

设 n_h 为高度为 h 的平衡二叉树的最小结点数. 二叉树看起来应该是如下结构形式:



斐波那契序列:

$$F_0 = 1, F_1 = 1, F_i = F_{i-1} + F_{i-2} \text{ for } i > 1$$

$$\Rightarrow n_h = F_{h+2} - 1, \text{ for } h \geq 0$$

斐波那契序列的理论值是:

$$F_i \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^i$$

$$\Rightarrow n_h \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{h+2} - 1$$

$$\Rightarrow h = O(\ln n)$$

设 n_h 是高度为 h 的平衡二叉树的最小结点数. $\Rightarrow n_h = n_{h-1} + n_{h-2} + 1$

h	n_h	F_h
0	1	1
1	2	1
2	4	2
3	7	3
4	12	5
5	20	8
6	33	13
7	54	21
8	88	34
9		

$$\Rightarrow n_h = F_{h+2} - 1, \quad (\text{对 } h \geq 0)$$

$$\Rightarrow n_h \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{h+2} - 1$$

$$\Rightarrow h = O(\ln n)$$

➤ 给定结点数为 n 的AVL树的最大高度为 $O(\log_2 n)$!

➤ 从而保证了AVL树的查找时间性能为 $O(\log_2 n)$!